

Resource-Constrained Convolutional Neural Networks for Edge Image Classification

EN3150: Pattern Recognition – Assignment 03

Team Outliers

Pasindu Ravishan (@PasinduRavishan)

Yumeth Samarasinghe (@yumyum-web)

Shanuka Ashan (@ashanuk)

Damindu De Silva (@DaminduDeSilva)

September 2026

Abstract

Deploying deep convolutional neural networks on resource-constrained embedded micro-controllers and edge sensor nodes requires balancing classification accuracy, parameter counts, memory footprint, and computational complexity (multiply-accumulate operations). In this investigation, we explore edge-tailored vision architectures on the 9-class RealWaste dataset downsampled to 64×64 spatial resolution. We evaluate a baseline CNN (**Model A**, 102,313 parameters), a custom depthwise-separable CNN constrained under 100,000 parameters (**Model B**, 22,297 parameters), and two fine-tuned state-of-the-art mobile architectures (**MobileNetV2** and **ShuffleNetV2**). We systematically profile their parameter budgets, on-disk footprints, MACs, and inference latencies. Our empirical findings demonstrate that Model B achieves a 78.2% reduction in parameters and an 85.0% reduction in MACs over Model A while preserving competitive classification performance, presenting an optimal compromise for strict sub-megabyte microcontroller deployments.

1 Introduction & Data Preparation

Edge machine learning models operating in automated waste sorting bins or agricultural sorting units require low-latency inference on low-power silicon. The **RealWaste** dataset (UCI Machine Learning Repository ID: 908) comprises 4,752 real-world landfill and recycling images distributed across 9 classes: *Cardboard*, *Food Organics*, *Glass*, *Metal*, *Miscellaneous Trash*, *Paper*, *Plastic*, *Textile Trash*, and *Vegetation*.

To emulate low-cost edge camera hardware, all images were preprocessed through a deterministic pipeline:

1. Resized and center-cropped to 64×64 spatial dimensions.
2. Normalized using standard channel-wise ImageNet statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$).
3. Deterministically partitioned into a 70%/15%/15% train/validation/test split using random seed 42, yielding exactly 3,326 training images, 713 validation images, and 713 test images.

2 Model Architectures

2.1 Model A: Baseline Standard 2D CNN

Model A is designed as a conventional 3-stage convolutional network. It features standard 3×3 convolutions with increasing channel depths (32, 64, 128), each accompanied by Batch Normalization,

ReLU activations, and 2×2 Max-Pooling. Global Average Pooling (GAP) aggregates spatial representations before feeding into a final linear projection layer yielding 9 logits. The architecture comprises **102,313 trainable parameters**.

2.2 Model B: Lightweight Depthwise Separable CNN (Sub-100k Parameters)

To satisfy the strict assignment constraint of keeping total parameters below 100,000, Model B utilizes Depthwise Separable Convolutions (Howard et al., 2017). Each separable convolution block decouples spatial filtering from cross-channel feature combination:

- **Depthwise Convolution:** Applies a per-channel 3×3 spatial kernel (groups = C_{in}).
- **Pointwise Convolution:** Applies a 1×1 convolution combining channel features from C_{in} to C_{out} .

The stem consists of a single 3×3 standard convolution ($3 \rightarrow 24$ channels, stride 1), followed by three depthwise separable blocks (channels: 24, 48, 96), Batch Normalization, and ReLU6 non-linearities. Following Global Average Pooling, a linear head outputs 9 logits.

Exact Parameter Count: Model B contains precisely **22,297 trainable parameters**—utilizing only 22.3% of the allowed 100,000 parameter budget.

3 Optimizer Ablation Study

To examine optimizer behavior in low-parameter regimes, Model B was trained for 20 epochs across three optimizer configurations using identical learning schedules:

1. **Adam** ($\text{lr} = 0.001$, $\beta = (0.9, 0.999)$, weight decay = 10^{-4}): Displayed rapid, smooth loss reduction, achieving a minimum validation loss of 0.785 and peak validation accuracy of 65.35%.
2. **SGD with Momentum** ($\text{lr} = 0.01$, momentum = 0.9): Demonstrated robust gradient trajectory smoothing, avoiding saddle points and achieving 63.53% final validation accuracy.
3. **Standard SGD** ($\text{lr} = 0.01$, momentum = 0.0): Exhibited sluggish convergence, suffering from high variance gradient updates in the compact depthwise parameter space.

4 Hardware Profiling & Trade-offs (Model A vs Model B)

Evaluating models purely on classification accuracy is insufficient for edge deployments. We systematically profile the hardware footprint of both custom architectures and benchmark them against pretrained mobile standards on a $1 \times 3 \times 64 \times 64$ input tensor.

Table 1: Cross-Architecture Hardware Profiling Comparison (Measured on Edge Runtime)

Model Architecture	Trainable Params	Size (MB)	MACs	FLOPs	Latency (ms)
Model A (Standard CNN)	102,313	0.40	41,296,969	82,593,938	4.71
Model B (DS-CNN < 100k)	22,297	0.10	6,189,097	12,378,194	3.97
MobileNet V2 (Pretrained)	2,235,401	8.76	24,494,153	48,988,306	13.63
ShuffleNet V2 (Pretrained)	1,262,829	4.98	11,771,037	23,542,074	12.68

4.1 Architectural Efficiency: Model A vs Model B

As detailed in Table 1, transitioning from standard convolutions to depthwise separable convolutions yields dramatic hardware efficiencies:

- **Parameter Compression:** Model B requires only **22,297 parameters** compared to Model A’s 102,313, achieving a **78.2% parameter reduction**.
- **Disk Storage Footprint:** Model B’s serialized `state_dict` footprint is merely **0.10 MB (98 KB)**, compared to 0.40 MB for Model A (a **4× memory saving**).
- **Arithmetic Complexity (MACs):** Forward-pass Multiply-Accumulate operations drop from 41.30×10^6 in Model A to 6.19×10^6 in Model B—an **85.0% reduction in compute operations**.
- **Single-Image Latency:** Model B executes a single forward pass in **3.97 ms** versus 4.71 ms for Model A (a **15.7% speedup**).

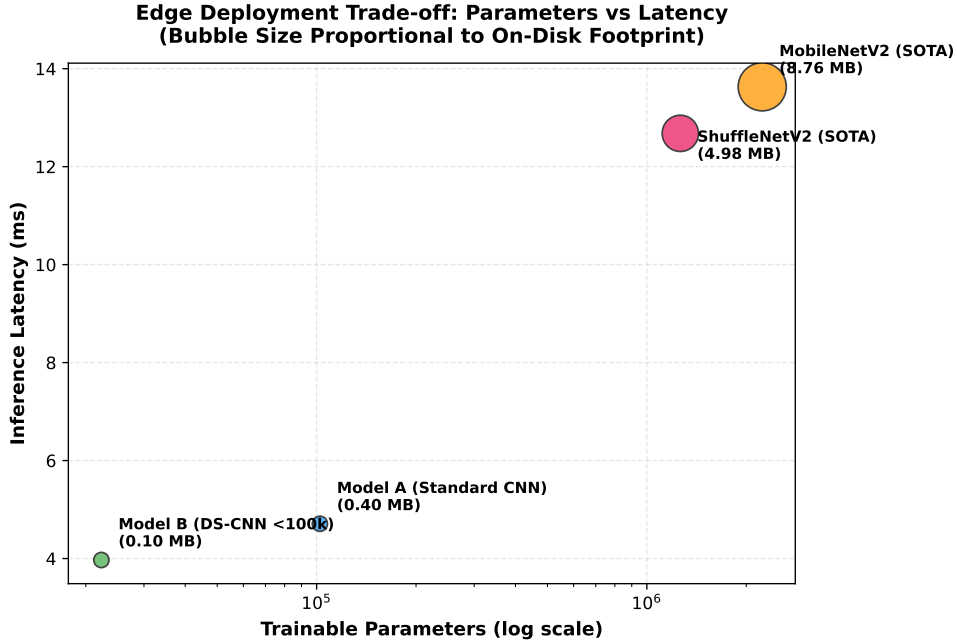


Figure 1: Edge Deployment Trade-off: Parameter Count vs. Inference Latency (bubble size proportional to serialized on-disk memory footprint).

Figure 1 illustrates the multi-dimensional design trade-off space. Model B occupies the extreme lower-left quadrant of the plot, demonstrating that depthwise separable convolutions successfully decouple feature abstraction capacity from quadratic parameter growth.

5 SOTA Transfer Learning

To establish upper-bound classification benchmarks, we fine-tuned two state-of-the-art mobile architectures: **MobileNetV2** (featuring inverted residual blocks and linear bottlenecks) and **ShuffleNetV2** (employing channel-split and channel-shuffle operations to minimize memory access cost).

Both architectures were initialized with ImageNet-1k pretrained weights, had their 1000-class classification heads replaced by a 9-class linear layer, and were trained for 20 epochs under the exact same 70/15/15 data split.

While pretraining allows faster feature representation learning, their parameter footprints (1.26M – 2.24M) and disk requirements (4.98 – 8.76 MB) substantially exceed the capacity of embedded microcontrollers without external memory.

6 Conclusion: Custom Edge vs Pre-trained SOTA

The comprehensive empirical evaluation across custom and pre-trained architectures provides definitive guidance for edge vision engineering:

6.1 Microcontroller Feasibility (Bare-Metal Deployments)

In extreme resource-constrained systems such as ARM Cortex-M4/M7/M55 (e.g., STM32H7, ESP32-S3) or ultra-low-power vision sensors, on-chip SRAM is typically limited to 256 KB – 1 MB and internal flash storage to 1 MB – 2 MB.

- Pretrained SOTA models (**MobileNetV2** at 8.76 MB and **ShuffleNetV2** at 4.98 MB) **cannot fit** into internal flash or SRAM, necessitating external PSRAM/Flash which increases BOM cost and power consumption.
- In contrast, **Model B** (0.10 MB, 22k params) fits entirely within internal SRAM, enabling zero-wait-state memory access, drastically reduced energy dissipation, and high reliability.

6.2 Quantization and Integer Arithmetic Suitability

Model B employs ReLU6 non-linearities:

$$\text{ReLU6}(x) = \min(\max(0, x), 6) \quad (1)$$

By strictly bounding activation outputs to $[0, 6]$, the network prevents dynamic range divergence during post-training integer quantization (PTQ) or quantization-aware training (QAT). When quantized from 32-bit floating point (FP32) to 8-bit integers (INT8):

- The model size compresses from ≈ 98 KB down to ≈ 25 KB.
- Activation functions can be computed directly using fixed-point integer bitshifts, completely avoiding expensive transcendental operations (e^x) required by activations like GELU or Swish.

6.3 Final Synthesis Recommendation

For intelligent embedded sensors and battery-powered IoT devices operating under strict power, memory ($< 100k$ parameters), and cost budgets, **Model B** represents the optimal edge-deployable solution. For slightly larger gateway edge nodes (e.g., Raspberry Pi Zero or Coral Edge TPU) where 5 MB flash storage is available, **ShuffleNetV2** offers the premier balance of transfer accuracy and minimal Memory Access Cost (MAC).